

Table 6. Optimized layouts configured by Def_{opt} , $PTO_{HumanExp}$ and PTO for web_returns and web_sales in TPC-DS SF 30K.

Table	Total Size	Def_{opt}			$PTO_{HumanExp}$			PTO		
		Partitioning Column	TFS (MB)	RGS (MB)	Sort Scheme	Partitioning Column	TFS (MB)	RGS (MB)	Sort Scheme	Sort Scheme
web_returns	130.6 GB	-	28	28	binpack	wr_returned_date_sk	64	16	wr_refunded_addr_sk	wr_refunded_addr_sk
web_sales	1.15 TB	ws_sold_date_sk	43	43	binpack	ws_sold_date_sk	103	26	z-order(ws_ship_date_sk, ws_ship_hdemo_sk)	

Table 7. Latency breakdown for TPC-DS SF 30K.

Table	# Scan Fragment Queries	Sort cluster size estimation	Target label estimation (sample)		Apply optimized layout
			Vanilla PTO	Lightweight PTO (# Training queries)	
web_returns	10	4,857 sec	5,593 sec	2,858 sec (4 queries)	300 sec
web_sales	35	7,119 sec	18,725 sec	9,569 sec (16 queries)	3,360 sec

Table 8. Sample Scan Fragment Query Latencies (TPC-DS SF 30K)

Table	Scan Fragment Query	Def_{opt}		PTO
		HumanExp	HumanExp	
web_returns	SELECT count(*),sum(wr_return_quantity) FROM WEB_RETURNS WHERE wr_returned_date_sk BETWEEN 2452133 AND 2452163;	6.7 sec	2.5 sec	2.5 sec
web_sales	SELECT count(*),sum(ws_quantity) FROM WEB_SALES WHERE ((ws_web_page_sk BETWEEN 6 AND 2966) AND (ws_ship_hdemo_sk BETWEEN 240 AND 6359) AND (ws_sold_time_sk BETWEEN 32400 AND 39599));	109.2 sec	60 sec	14.5 sec

counter the downscaling effect. This specific anomaly was rectified at SF 30K where the absolute sizes of the samples were larger and TFS and RGS were pushed to higher values only if necessary and not because samples were too small to yield substantial #row groups or files for a particular layout configuration. Another interesting observation is that the sort scheme changed at SF 30K from what was earlier discovered at SF 10K. This can be explained by the fact that the cardinality of unique keys and concentration of data skews do not grow linearly between scale factors although the row counts scale linearly for fact tables. Additionally, the absolute sample sizes changed at SF 30K along with the estimated skipping benefits for the training layouts which led to GBT predicting different skipping benefits (regression scores) for the candidate layouts compared to SF 10K.

Besides altering the TFS candidate space, we implemented a Lightweight PTO that used only a subset of scan fragment queries to estimate skipping benefits (training labels). We chose scan fragment queries having distinct combinations of columns in their filter predicates and this led to training on 4 out of 10 scan fragment queries for web_returns and 16 out of 35 scan fragment queries for web_sales. This optimization reduced the training label estimation latency by ~50% as shown in Table 7 without sacrificing layout quality. Figure 10 shows that PTO reduced workload latency by 21% on web_returns and 79% on web_sales over *DefOpt*. Interestingly, there were no regressions on web_sales despite training only on a fraction of the queries and all the scan fragment queries were benefitted by 40%-93%. On web_returns, 5 out of 10 queries were benefitted by 26%-62% and the regressed queries did not benefit from the sort scheme due to a non-overlapping filter column. $PTO_{HumanExp}$ found the same TFS, RGS as PTO on web_returns and benefitted same #queries but detected lower TFS, RGS for web_sales thereby incurring longer workload latencies and establishing the need to re-discover human expert heuristics when scale factors (and datasets) vary. Table 8 shows latencies for sample queries.

5 Discussion

In this section, we clearly state the scope of PTO and the scenarios where it will be applicable in its current implementation while discussing its possible extensions.

1. Generalizability: PTO is applicable to lakehouse systems and open table formats with similar data layouts (parameterized by file sizes, partitioning, sort scheme) and can be adapted to other table formats as long as they have tunable parameters. Micropartitions in Snowflake [14] eliminate the need to detect a partition column but require periodical re-clustering with manually specified columns and MAX_SIZE to maintain a desired system clustering ratio. Automatic clustering [22] in Snowflake decides when to run re-clustering but does not identify the sort columns. PTO is still relevant in such cases to minimize manual intervention. Likewise, Databricks liquid clustering [20, 30] does not partition tables but configures clustering columns, MIN_CUBE_SIZE and TARGET_CUBE_SIZE. Although a separate implementation for each lakehouse system is beyond the scope of this paper, ideas from PTO like reducing search space of candidate layouts and training ML models on skipping benefits learned from sampled tables can be applied to other systems.

2. Scalability w.r.t. #Tables: Since there are multiple tables in a lakehouse, it may not be scalable for PTO to run table optimization for each table. We currently prioritize tables to be optimized based on their corresponding sample sizes. If we end up with empty samples for a table, it is left unoptimized. A possible extension is that PTO can prioritize the tables in a lakehouse w.r.t. their cardinalities, schema width in terms of #frequently queried columns a table contains, #queries in which a table appears and their frequency. We can let PTO run either until a pre-specified latency budget is exhausted or all the tables are optimized. It is likely that the budget is exhausted well before all the tables are optimized and the remaining tables which are left unoptimized are hopefully less important to the workload. Additionally, PTO can be applied in parallel to multiple tables given a larger computational cluster with enough storage as tables can be optimized independent of each

other. All the steps in PTO such as table sampling, estimating the skipping benefits for shortlisted candidate layouts on sampled tables, ML model training and inference can be run in parallel across tables. We apply PTO sequentially to each table due to resource constraints.

3. Optimization granularity: PTO currently detects optimal layouts at a table level and can be extended to support partition-level optimization. This is cheaper than rewriting the entire table when data ingests affect a subset of the partitions but this also assumes that the partitioning scheme cannot change after data ingests and is hence not re-discovered. PTO can achieve partition-level rewrites by leveraging the support for WHERE clauses during rewrites offered by the Spark-Iceberg library [5]. We can specify the partitions to be optimized using the `rewrite_data_files` procedure in a selection predicate on the partitioning column.

4. Data Ingests and Workload Evolution: We will discuss the following scenarios separately - catering to data ingests and query workload drift in isolation vs. handling workload drift while data ingests are concurrently happening. Data ingestion and workload drift are tackled separately in PTO. Data ingests can change the underlying data distribution and result in stale table samples which necessitate re-sampling and re-learning the skipping benefits. To support data ingests, the current implementation of PTO is re-run periodically in a batched manner on the entire table. This can be further optimized by applying PTO only on the affected partitions as mentioned earlier thereby isolating the rewrites from concurrent data ingests. A tradeoff here is that simultaneous data ingests can create new candidates for layout parameters such as a larger TFS and corresponding RGS. We do not expect new candidate sort schemes unless new queries arrive with filter predicates on columns outside the candidate list or schema-level changes are done introducing new columns. Since we apply PTO in a batched manner, we can support offline partition-level optimization but supporting concurrent data ingests in real time requires careful re-design.

On the other hand, supporting workload evolution in isolation is more straightforward. Although automatic workload drift detection is not supported, we can invoke PTO explicitly when new queries arrive. PTO can be retrained on an updated workload containing all the latest queries and a tiny subset of older queries having the highest weights (Section 4.3.4 discusses query weighting).

Supporting concurrent data ingests while query workload evolves is a challenging problem because data ingestion necessitates re-learning of skipping benefits even for historical (repeating) candidate layouts. That is possible when the workload is considered static and allows estimating the target labels on a *frozen* set of scan fragments. If the scan fragments are evolving concurrently as ingests happen, we pivot the learning to the most recent workload.

6 Related Work

As data gets ingested into the Parquet (or ORC or any lakehouse-compatible columnar formats) files within the Iceberg format, multiple small files are created. This coupled with updates via Copy-on-Write (CoW) and Merge-on-Read (MoR) techniques [28] entails that periodic compaction is applied on small files to get larger Parquet files meeting the requirement of a pre-specified target file size (TFS). The compacted files are sized such that the individual file size is close to TFS. AutoComp [25] analyzes the cost-benefit tradeoff in compacting candidate small files based on multiple heuristics such as the rewrite costs and the computational resources spent in reaching a reduced file count vs. the table utility in a given workload. AutoComp further enables parallel rewrites by scoping the rewrite granularity at partition-level or snapshot-level. AutoComp also suggests that TFS could be tuned based on the query workload. While the OPTIMIZE feature in Dremio [12, 13], Delta Lake [17], and S3 Tables [7] achieve compaction, they require TFS to be manually specified or a default value of TFS is applied in all the cases. Similarly, a default row group size (RGS) is used to create the row groups within each Parquet file. Predictive optimization

in Databricks Delta Lake [18] determines when to periodically re-run the OPTIMIZE command but does not discover the table optimization parameters.

At the time of querying, skipping of irrelevant data is maximized when the query predicates match the partitioning scheme and the sort strategy applied on the tables. Zimmerer et al. [43] extend the scope of pruning techniques for Snowflake micro-partitions beyond filtered scan predicates by deriving additional min/max ranges for complex expressions involving LIMIT, top-k and JOIN operators. Such advanced pruning techniques can be augmented with PTO to further enhance the skipping capability. Amazon Redshift [19] exploits the multidimensional predicates (MDDL) appearing in the query workload to determine the sort order but does not automatically detect the partitioning columns, TFS or RGS. Another limitation of MDDL is that the predicates used to sort the columns require an exact match with the query predicates to yield any skipping benefits. Also, the predicates containing non-deterministic functions such as RANDOM() are not used for sort ordering. PTO, on the other hand, does not make any such assumptions about the query predicates as the sorting is done on columns. Similarly to MDDL, we re-run PTO periodically to support workload evolution and data ingests. On the flipside, PTO assumes access to both data and query workload whereas MDDL does not need access to underlying data thereby preserving data privacy.

HP Vertica DBDesigner [39] discovers sort strategies by including candidate sort columns from join predicates, aggregates and GROUP BY clauses besides scan predicates which enriches the candidate search space. However, it does not apply Z-ordering and sort orders are discovered for materialized views. Databricks liquid clustering [20, 30] discovers optimal layouts and differs from PTO w.r.t. the space filling curve (Hilbert vs. Z-ordering). It is amenable to data updates because it avoids the need to specify a partitioning column but configures clustering columns, MIN_CUBE_SIZE and TARGET_CUBE_SIZE. Liquid clustering is not open-sourced for Iceberg tables that precludes a comparison with PTO. Micropartitions in Snowflake [14] require clustering columns and MAX_SIZE to be configured which can benefit from PTO.

7 Conclusion

In this work, we built a workload-driven predictive table optimizer, PTO for Apache Iceberg tables on the top of Presto lakehouse engine. PTO leverages the scan fragments in the workload to co-discover the optimal partitioning column, target file size, row group size and the multidimensional sort strategy. We propose heuristics to significantly reduce the candidate search space based on sort cluster size, selectivity and filter column frequency of the columns appearing in the filter predicates of the scan fragments. Subsequently, we learn a predictive model (GBT) utilizing as few as ~3-4% of the candidate space to reduce the layout discovery latency significantly and estimate the data skipping benefits of the candidate layouts. The optimal layout predicted by the ML model is applied on the full set of tables. Our experimental results on TPC-H and TPC-DS at SF 10K show that PTO yields up to 11%-36% average savings in workload execution latencies while speeding up scan-intensive, long latency queries by 3.4× - 11×. Our evaluation against multiple baselines validates the design choices for each component in PTO w.r.t. balancing the optimality of layouts and #queries benefited with the layout discovery latency. Currently, we support evolving query workload and data by periodically re-running PTO and a more principled solution for these cases is left for future work.

References

- [1] 2017. Apache Hudi. <https://hudi.apache.org/docs/overview>.
- [2] 2017. Apache Hudi Github Repository. <https://github.com/apache/hudi>.
- [3] 2017. Apache Iceberg Documentation. <https://iceberg.apache.org/docs/nightly/>.
- [4] 2017. Apache Iceberg Github Repository. <https://github.com/apache/iceberg>.

- [5] 2017. Apache Iceberg: Spark procedures. <https://iceberg.apache.org/docs/nightly/spark-procedures/>.
- [6] Michael Armbrust, Tathagata Das, Sameer Paranjpye, Reynold Xin, Shixiong Zhu, Ali Ghodsi, Burak Yavuz, Mukul Murthy, Joseph Torres, Liwen Sun, Peter A. Boncz, Mostafa Mokhtar, Herman Van Hovell, Adrian Ionescu, Alicja Luszczak, Michal Switakowski, Takuya Ueshin, Xiao Li, Michal Szafranski, Pieter Senster, and Matei Zaharia. 2020. Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores. *Proc. VLDB Endow.* 13, 12 (2020), 3411–3424. doi:10.14778/3415478.3415560
- [7] Jeff Barr. 2024. New Amazon S3 Tables: Storage optimized for analytics workloads. <https://aws.amazon.com/blogs/aws/new-amazon-s3-tables-storage-optimized-for-analytics-workloads/>.
- [8] James Bergstra, Daniel Yamins, and David D. Cox. 2013. Hyperopt: Distributed Hyperparameter Optimization. <https://github.com/hyperopt/hyperopt/>.
- [9] James Bergstra, Daniel Yamins, and David D. Cox. 2013. Making a Science of Model Search: Hyperparameter Optimization in Hundreds of Dimensions for Vision Architectures. In *Proceedings of the 30th International Conference on Machine Learning, ICML 2013, Atlanta, GA, USA, 16-21 June 2013 (JMLR Workshop and Conference Proceedings, Vol. 28)*. JMLR.org, 115–123. <http://proceedings.mlr.press/v28/bergstra13.html>
- [10] Peter A. Boncz, Thomas Neumann, and Orri Erling. 2013. TPC-H Analyzed: Hidden Messages and Lessons Learned from an Influential Benchmark. In *Performance Characterization and Benchmarking - 5th TPC Technology Conference, TPCTC 2013, Trento, Italy, August 26, 2013, Revised Selected Papers (Lecture Notes in Computer Science, Vol. 8391)*, Raghunath Nambiar and Meikel Poess (Eds.). Springer, 61–76. doi:10.1007/978-3-319-04936-6_5
- [11] Ramesh Chandra, Haogang Chen, Ray Matharu, Sarah Cai, Jeff Chen, Priyam Dutta, Bogdan Ghita, Todd Greenstein, Gopal Holla, Peng Huang, Yuchen Huo, Adrian Ionescu, Adriana Ispas, Tim Januschowski, Vihang Karajgaonkar, Stefania Leone, David Lewis, Andrew Li, Nong Li, Cheng Lian, Stephen Link, Qing Lu, Yesheng Ma, Chris Pettitt, Vijayan Prabhakaran, Bogdan Raducanu, Kyle Rong, Paul Roome, Samarth Shetty, Sean Smith, Xiaotong Sun, Yuyuan Tang, Weitao Wen, Lei Xia, Junlin Zeng, Ben Zhang, Reynold Xin, and Matei Zaharia. 2025. Unity Catalog: Open and Universal Governance for the Lakehouse and Beyond. In *Companion of the 2025 International Conference on Management of Data, SIGMOD/PODS 2025, Berlin, Germany, June 22-27, 2025*, Volker Markl, Joseph M. Hellerstein, and Azza Abouzied (Eds.). ACM, 310–322. doi:10.1145/3722212.3724459
- [12] Dremio Cloud. 2024. Optimize Table. <https://docs.dremio.com/cloud/reference/sql/commands/optimize-table/>.
- [13] Dremio Cloud. 2024. Optimizing Tables. <https://docs.dremio.com/cloud/sonar/query-manage/managing-tables/optimizing/>.
- [14] Snowflake Community. 2022. Understanding Micro-partitions and Data Clustering. <https://community.snowflake.com/s/article/understanding-micro-partitions-and-data-clustering>.
- [15] Transaction Processing Council. 1999. TPC-H Version 2 and Version 3. <https://www.tpc.org/tpch/>.
- [16] Transaction Processing Council. 2006. TPC-DS Version 2 and Version 3. <https://www.tpc.org/tpcds/>.
- [17] Databricks. 2024. Optimize data file layout. <https://docs.databricks.com/en/delta/optimize.html>.
- [18] Databricks. 2024. Predictive optimization for Unity Catalog managed tables. <https://docs.databricks.com/en/optimizations/predictive-optimization.html>.
- [19] Jialin Ding, Matt Abrams, Sanghita Bandyopadhyay, Luciano Di Palma, Yanzhu Ji, Davide Pagano, Gopal Paliwal, Panos Parchas, Pascal Pfeil, Orestis Polychroniou, Gaurav Saxena, Aamer Shah, Amina Voloder, Sherry Xiao, Davis Zhang, and Tim Kraska. 2024. Automated Multidimensional Data Layouts in Amazon Redshift. In *Companion of the 2024 International Conference on Management of Data, SIGMOD/PODS 2024, Santiago AA, Chile, June 9-15, 2024*, Pablo Barceló, Nayat Sánchez-Pi, Alexandra Meliou, and S. Sudarshan (Eds.). ACM, 55–67. doi:10.1145/3626246.3653379
- [20] Databricks Documentation. 2024. Use liquid clustering for Delta tables. <https://docs.databricks.com/en/delta/clustering.html>.
- [21] Presto 0.290 Documentation. 2024. Approximate Aggregate Functions. <https://prestodb.io/docs/current/functions/aggregate.html>.
- [22] Snowflake Documentation. 2022. Automatic Clustering. <https://docs.snowflake.com/en/user-guide/tables-auto-reclustering>.
- [23] Dremio. 2020. Project Nessie: Transactional Catalog for Data Lakes with Git-like semantics. <https://projectnessie.org>.
- [24] Dremio. 2025. The Architect’s Guide to Open Table Formats and Object Storage. <https://www.dremio.com/newsroom/the-architects-guide-to-open-table-formats-and-object-storage/>.
- [25] Anja Gruenheid, Jesús Camacho-Rodríguez, Carlo Curino, Raghu Ramakrishnan, Stanislav Pak, Sumedh Sakdeo, Lenisha Gandhi, Sandeep K. Singhal, Pooja Nilangekar, and Daniel J. Abadi. 2025. AutoComp: Automated Data Compaction for Log-Structured Tables in Data Lakes. In *Companion of the 2025 International Conference on Management of Data, SIGMOD/PODS 2025, Berlin, Germany, June 22-27, 2025*, Volker Markl, Joseph M. Hellerstein, and Azza Abouzied (Eds.). ACM, 404–417. doi:10.1145/3722212.3724430
- [26] Apache Hudi. 2017. Indexing. https://hudi.apache.org/docs/metadata_indexing/.

- [27] Jason Hughes. 2021. Apache Iceberg: An Architectural Look Under the Covers. <https://www.dremio.com/resources/guides/apache-iceberg-an-architectural-look-under-the-covers/>.
- [28] Paras Jain, Peter Kraft, Conor Power, Tathagata Das, Ion Stoica, and Matei Zaharia. 2023. Analyzing and Comparing Lakehouse Storage Systems. In *13th Conference on Innovative Data Systems Research, CIDR 2023, Amsterdam, The Netherlands, January 8-11, 2023*. www.cidrdb.org. <https://www.cidrdb.org/cidr2023/papers/p92-jain.pdf>
- [29] David A. McAllester and Robert E. Schapire. 2000. On the Convergence Rate of Good-Turing Estimators. In *Proceedings of the Thirteenth Annual Conference on Computational Learning Theory (COLT 2000), June 28 - July 1, 2000, Palo Alto, California, USA*, Nicolò Cesa-Bianchi and Sally A. Goldman (Eds.). Morgan Kaufmann, 1–6.
- [30] jaseidman mssaperla, irinaskaya. 2024. Use liquid clustering for delta tables. <https://learn.microsoft.com/en-us/azure/databricks/delta/clustering>.
- [31] Raghunath Othayoth Nambiar and Meikel Poess. 2006. The Making of TPC-DS. In *Proceedings of the 32nd International Conference on Very Large Data Bases, Seoul, Korea, September 12-15, 2006*, Umeshwar Dayal, Kyu-Young Whang, David B. Lomet, Gustavo Alonso, Guy M. Lohman, Martin L. Kersten, Sang Kyun Cha, and Young-Kuk Kim (Eds.). ACM, 1049–1058. <http://dl.acm.org/citation.cfm?id=1164217>
- [32] Alon Orlitsky and Ananda Theertha Suresh. 2015. Competitive Distribution Estimation: Why is Good-Turing Good. In *Advances in Neural Information Processing Systems 28: Annual Conference on Neural Information Processing Systems 2015, December 7-12, 2015, Montreal, Quebec, Canada*, Corinna Cortes, Neil D. Lawrence, Daniel D. Lee, Masashi Sugiyama, and Roman Garnett (Eds.). 2143–2151. <https://proceedings.neurips.cc/paper/2015/hash/d759175de8ea5b1d9a2660e45554894f-Abstract.html>
- [33] Pedro Pedreira, Orri Erling, Maria Basmanova, Kevin Wilfong, Laith Sakka, Krishna Pai, Wei He, and Biswapesh Chattopadhyay. 2022. Velox: Meta’s Unified Execution Engine. *Proc. VLDB Endow.* 15, 12 (2022), 3372–3384. doi:10.14778/3554821.3554829
- [34] Raghav Sethi, Martin Traverso, Dain Sundstrom, David Phillips, Wenlei Xie, Yutian Sun, Nezh Yegitbasi, Haozhun Jin, Eric Hwang, Nileema Shingte, and Christopher Berner. 2019. Presto: SQL on Everything. In *35th IEEE International Conference on Data Engineering, ICDE 2019, Macao, China, April 8-11, 2019*. IEEE, 1802–1813. doi:10.1109/ICDE.2019.00196
- [35] Evan Smith. 2024. What are Open Table Formats? Why data lakehouses rely on open table formats like Iceberg. <https://www.starburst.io/blog/open-table-formats/>.
- [36] Simon Spati. 2023. The Open Lakehouse Stack: DuckDB and the Rise of Table Formats. <https://motherduck.com/blog/open-lakehouse-stack-duckdb-table-formats/>.
- [37] Yutian Sun, Tim Meehan, Rebecca Schlusell, Wenlei Xie, Masha Basmanova, Orri Erling, Andrii Rosa, Shixuan Fan, Rongrong Zhong, Arun Thirupathi, Nikhil Collooru, Ke Wang, Sameer Agarwal, Arjun Gupta, Dionysios Logothetis, Kostas Xirogiannopoulos, Amit Dutta, Varun Gajjala, Rohit Jain, Ajay Palakuzhy, Prithvi Pandian, Sergey Pershin, Abhisek Saikia, Pranjal Shankhdhar, Neerad Somanchi, Swapnil Tailor, Jialiang Tan, Sreeni Viswanadha, Zac Wen, Biswapesh Chattopadhyay, Bin Fan, Deepak Majeti, and Aditi Pandit. 2023. Presto: A Decade of SQL Analytics at Meta. *Proc. ACM Manag. Data* 1, 2 (2023), 189:1–189:25. doi:10.1145/3589769
- [38] Ashish Thusoo, Joydeep Sen Sarma, Namit Jain, Zheng Shao, Prasad Chakka, Suresh Anthony, Hao Liu, Pete Wyckoff, and Raghootham Murthy. 2009. Hive - A Warehousing Solution Over a Map-Reduce Framework. *Proc. VLDB Endow.* 2, 2 (2009), 1626–1629. doi:10.14778/1687553.1687609
- [39] Ramakrishna Varadarajan, Vivek Bharathan, Ariel Cary, Jaimin Dave, and Sreenath Bodagala. 2014. DBDesigner: A customizable physical design tool for Vertica Analytic Database. In *IEEE 30th International Conference on Data Engineering, Chicago, ICDE 2014, IL, USA, March 31 - April 4, 2014*, Isabel F. Cruz, Elena Ferrari, Yufei Tao, Elisa Bertino, and Goce Trajcevski (Eds.). IEEE Computer Society, 1084–1095. doi:10.1109/ICDE.2014.6816725
- [40] Kyle Weller. 2024. Comprehensive Data Catalog Comparison. <https://www.onehouse.ai/blog/comprehensive-data-catalog-comparison>.
- [41] Matei Zaharia, Ali Ghodsi, Reynold Xin, and Michael Armbrust. 2021. Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics. In *11th Conference on Innovative Data Systems Research, CIDR 2021, Virtual Event, January 11-15, 2021, Online Proceedings*. www.cidrdb.org. http://cidrdb.org/cidr2021/papers/cidr2021_paper17.pdf
- [42] Xinyu Zeng, Yulong Hui, Jiahong Shen, Andrew Pavlo, Wes McKinney, and Huanchen Zhang. 2023. An Empirical Evaluation of Columnar Storage Formats. *Proc. VLDB Endow.* 17, 2 (2023), 148–161. <https://www.vldb.org/pvldb/vol17/p148-zeng.pdf>
- [43] Andreas Zimmerer, Damien Dam, Jan Kossmann, Juliane Waack, Ismail Oukid, and Andreas Kipf. 2025. Pruning in Snowflake: Working Smarter, Not Harder. In *Companion of the 2025 International Conference on Management of Data, SIGMOD/PODS 2025, Berlin, Germany, June 22-27, 2025*, Volker Markl, Joseph M. Hellerstein, and Azza Abouzied (Eds.). ACM, 757–770. doi:10.1145/3722212.3724447

Received July 2025; revised October 2025; accepted November 2025